



Comandos

Cree comandos personalizados para tareas repetitivas.

Los comandos personalizados le permiten especificar un mensaje que desea ejecutar cuando ese comando se ejecuta en TUI.

```
/my-command
```

Los comandos personalizados se suman a los comandos integrados como ``/init``, ``/undo``, ``/redo``, ``/share``, ``/help``. [Más información](#).

Crear archivos de comando

Cree archivos de Markdown en el directorio ``commands/`` para definir comandos personalizados.

Crear ``.opencode/commands/test.md``:

```
.opencode/commands/test.md
```

```
---
description: Run tests with coverage
agent: build
model: anthropic/claude-3-5-sonnet-20241022
---
```

Run the full test suite with coverage report and show any failures.
Focus on the failing tests and suggest fixes.

El frontmatter define las propiedades del comando. El contenido se convierte en la plantilla.

Utilice el comando escribiendo ``/`` seguido del nombre del comando.

```
"/test"
```

Configuración

Puede agregar comandos personalizados a través de la configuración OpenCode o creando archivos de Markdown en el directorio ``commands/``.

JSON

Utilice la opción ``command`` en su OpenCode config:

opencode.jsonc

```
{
  "$schema": "https://opencode.ai/config.json",
  "command": {
    // This becomes the name of the command
    "test": {
      // This is the prompt that will be sent to the LLM
      "template": "Run the full test suite with coverage report and show any
failures.\nFocus on the failing tests and suggest fixes.",
      // This is shown as the description in the TUI
      "description": "Run tests with coverage",
      "agent": "build",
      "model": "anthropic/claude-3-5-sonnet-20241022"
    }
  }
}
```

Ahora puedes ejecutar este comando en TUI:

```
/test
```

Markdown

También puede definir comandos utilizando archivos de Markdown. Colócalos en:

Global: `~/ .config/opencode/commands/``

Por proyecto: ``.opencode/commands/``

```
~/ .config/opencode/commands/test.md
```

```
---
```

```
description: Run tests with coverage
```

```
agent: build
```

```
model: anthropic/claude-3-5-sonnet-20241022
```

```
---
```

```
Run the full test suite with coverage report and show any failures.  
Focus on the failing tests and suggest fixes.
```

El nombre del archivo de Markdown se convierte en el nombre del comando. Por ejemplo, ``test.md`` permite que lo ejecute:

```
/test
```

Configuración de indicaciones

Las indicaciones para los comandos personalizados admiten varios marcadores de posición y sintaxis especiales.

Argumentos

Pase argumentos a comandos usando el marcador de posición ``$ARGUMENTS``.

```
.opencode/commands/component.md
```

```
---
```

```
description: Create a new component
```

```
---
```

```
Create a new React component named $ARGUMENTS with TypeScript support.  
Include proper typing and basic structure.
```

Ejecute el comando con argumentos:

```
/component Button
```

Y ``$ARGUMENTS`` será reemplazado por ``Button``.

También puede acceder a argumentos individuales utilizando parámetros posicionales:

``$1`` - Primer argumento

``$2`` - Segundo argumento

``$3`` - Tercer argumento

Etcétera...

Por ejemplo:

```
.opencode/commands/create-file.md
```

```
---
```

```
description: Create a new file with content
```

```
---
```

```
Create a file named $1 in the directory $2  
with the following content: $3
```

Ejecute el comando:

```
/create-file config.json src "{ \"key\": \"value\" }"
```

Esto reemplaza:

```
`$1` con `config.json`
```

```
`$2` con `src`
```

```
`$3` con `{ "key": "value" }`
```

Salida del shell

Utilice `!command` para inyectar la salida de comando bash en su mensaje.

Por ejemplo, para crear un comando personalizado que analice la cobertura de la prueba:

.opencode/commands/analyze-coverage.md	
--	--

description: Analyze test coverage

Here are the current test results:

`!`npm test``

Based on these results, suggest improvements to increase coverage.

0 para revisar cambios recientes:

.opencode/commands/review-changes.md	
--------------------------------------	--

description: Review recent changes

Recent git commits:

`!`git log --oneline -10``

Review these changes and suggest any improvements.

Los comandos se ejecutan en el directorio raíz de su proyecto y su salida pasa a formar parte del mensaje.

Referencias de archivos

Incluya archivos en su comando usando `@` seguido del nombre del archivo.

```
.opencode/commands/review-component.md
```

```
---
```

```
description: Review component
```

```
---
```

```
Review the component in @src/components/Button.tsx.  
Check for performance issues and suggest improvements.
```

El contenido del archivo se incluye automáticamente en el mensaje.

Opciones

Veamos las opciones de configuración en detalle.

Plantilla

La opción ``template`` define el mensaje que se enviará al LLM cuando se ejecute el comando.

```
opencode.json
```

```
{  
  "command": {  
    "test": {  
      "template": "Run the full test suite with coverage report and show any  
failures.\nFocus on the failing tests and suggest fixes."  
    }  
  }  
}
```

Esta es una opción de configuración **obligatoria**.

Descripción

Utilice la opción ``description`` para proporcionar una breve descripción de lo que hace el comando.

opencode.json	
---------------	--

```
{
  "command": {
    "test": {
      "description": "Run tests with coverage"
    }
  }
}
```

Esto se muestra como descripción en TUI cuando escribe el comando.

Agente

Utilice la configuración ``agent`` para especificar opcionalmente qué agente debe ejecutar este comando. Si se trata de un subagente, el comando activará una invocación de subagente de forma predeterminada. Para desactivar este comportamiento, establezca ``subtask`` en ``false``.

opencode.json

```
{
  "command": {
    "review": {
      "agent": "plan"
    }
  }
}
```

Esta es una opción de configuración **opcional**. Si no se especifica, el valor predeterminado es su agente actual.

Subtarea

Utilice el booleano ``subtask`` para forzar que el comando active una invocación de subagente. Esto es útil si desea que el comando no contamine su contexto principal y **obligue** al agente a actuar como subagente. incluso si ``mode`` está configurado en ``primary`` en la configuración de agente.

opencode.json

```
{
  "command": {
    "analyze": {
      "subtask": true
    }
  }
}
```

Esta es una opción de configuración **opcional**.

Modelo

Utilice la configuración ``model`` para anular el modelo predeterminado para este comando.

opencode.json

```
{
  "command": {
    "analyze": {
      "model": "anthropic/claude-3-5-sonnet-20241022"
    }
  }
}
```

Esta es una opción de configuración **opcional**.

Integrados

opencode incluye varios comandos integrados como ``/init``, ``/undo``, ``/redo``, ``/share``, ``/help``; [más información](#).

📌 NOTA

Los comandos personalizados pueden anular los comandos integrados.

Si define un comando personalizado con el mismo nombre, anulará el comando integrado.

 [Edita esta página](#)

 [Found a bug? Open an issue](#)

 [Join our Discord community](#)

© Anomaly

Última actualización: 21 may 2026

🚩 Español ▼